TRAIDORES - Manual técnico

Autor: Ignacio Giménez

Materia: Desarrollo de Sistemas Orientado a Objetos

Fecha de elaboración: 28 de octubre de 2026

Estado: documentación académica basada en las entregas disponibles; pendiente de contrastar con el código y la aplicación.

1. Objetivo y alcance

Este manual reúne el diseño técnico documentado de Traidores, juego móvil de deducción social, y orienta la construcción de una simulación académica de una partida en Python. El repositorio contiene documentos y una guía de programación; no contiene código fuente, ejecutable, dependencias ni configuración de despliegue. Por eso no es posible indicar comandos de instalación o ejecución verificados ni asegurar detalles internos de la aplicación móvil.

Hay dos alcances distintos: la documentación de análisis describe una beta Android con IA, partidas online, perfiles e historial; la guía de codificación propone una primera simulación en consola con decisiones preparadas, sin cuentas, red, chat, Firebase ni IA. La propuesta relacional con SQLite y SQLModel es académica y complementaria; el documento DER indica que el juego Android utiliza Firebase, no que haya sido migrado a SQLite.

2. Documentos de referencia

- analisis-y-diseno.pdf: historias, casos de uso y clases, versión del 26/08/2026.
- diagramas-de-secuencia.pdf: seis flujos propuestos, incluidos sala, partida, acción y votación; figura como borrador.
- diagramas-de-estados.pdf: estados propuestos de sala, partida, jugador y votación; figura como borrador.
- der-y-mapeo-orm.pdf: entidades, relaciones y propuesta de mapeo relacional del 13/10/2026.
- ../codificacion/Guia_para_programar_una_partida.md: reglas y orden de implementación de la simulación del 14/10/2026.

3. Modelo de dominio

Jugador representa la identidad de una persona. Sala agrupa jugadores, tiene un anfitrión y configura una partida. Partida conserva el ciclo, la ronda y la fase. Participacion vincula jugador, partida y rol y representa su estado temporal. Rol identifica equipo y habilidad. Resultado registra el cierre de una partida. El anfitrión es una relación con un jugador registrado y el jugador eliminado es un estado de participación, no una clase permanente.

En la simulación mínima la guía propone jugador.py, rol.py, participacion.py, partida.py y main.py. Se aconseja mantener las reglas en los objetos del dominio y usar main.py para construir el ejemplo, proporcionar decisiones y mostrar resultados. Los nombres y métodos de la guía son una propuesta, no archivos ya implementados.

4. Ciclo y reglas de la simulación

1. Crear doce jugadores y asignar 8 roles del Pueblo, 3 Traidores y 1 Neutral según la guía.
2. Registrar acciones nocturnas válidas de participantes vivos y resolver ataque, protección, investigación y restricción del Mercenario.
3. Comprobar victoria después de la noche. Si no hay ganador, pasar al debate y permitir la revelación del Alcalde.
4. Registrar un voto por participante habilitado y por votación. Resolver expulsión o desempate y comprobar la victoria.
5. Si la partida sigue, limpiar la restricción correspondiente al día y comenzar otra ronda.

Si Asesino y Espía eligen víctimas distintas, se sortea entre ellas; la protección del Médico puede evitar la muerte. El Detective ve al Espía como inocente. El Mercenario no puede elegirse a sí mismo ni repetir objetivo en noches consecutivas; el afectado no habla ni vota ese día. El Alcalde revelado tiene voto doble y decide entre los empatados; sin él se repite la votación entre empatados y, si vuelve a haber empate, nadie sale. El Bufón expulsado gana individualmente y la partida sigue.

El Pueblo gana si no quedan Asesinos ni Espías vivos, incluso si queda el Mercenario. Si queda alguno de esos dos roles, los Traidores ganan cuando igualan o superan a los participantes vivos del Pueblo. El Mercenario cuenta como Traidor y el Bufón no cuenta en ninguno de esos bandos. Comprobar primero la victoria del Pueblo.

Decisiones pendientes: orden exacto de las habilidades nocturnas; validez de la acción de quien muere esa noche; objetivos permitidos al votar, incluido el voto propio. No fijar estas reglas como definitivas sin acordarlas. La guía original contiene más detalle y prevalece para esta simulación.

5. Datos y persistencia propuestos

El DER incluye jugador, sala, partida, rol, participacion y resultado. Una sala tiene un anfitrión y puede generar varias partidas; una participación une un jugador, una partida y un rol; una partida tiene como máximo un resultado. El historial se obtiene de participaciones y resultados.

Para una implementación relacional académica se propone SQLModel con SQLite: claves primarias, claves foráneas y relaciones bidireccionales; unicidad de (jugador_id, partida_id) y de resultado.partida_id. Debe verificarse la integridad referencial y tratarse la información de cuenta de forma apropiada. Acciones, votos y chat no están incluidos en el modelo resumido; requerirían diseño adicional si se decidiera persistirlos. La simulación inicial no necesita base de datos.

6. Plan de implementación y comprobación

1. Implementar Jugador y Rol y verificar datos y equipos.
2. Implementar Participacion y verificar eliminación y habilitación para votar.
3. Crear Partida con doce participaciones y probar ambas condiciones de victoria, incluidos Mercenario y Bufón.
4. Implementar votos, voto doble y ambos tipos de desempate; comprobar duplicados y participantes inhabilitados.
5. Añadir acciones nocturnas y probar protección, investigación del Espía, restricciones del Mercenario y decisiones pendientes una vez acordadas.
6. Integrar las fases y el ejemplo de la guía: ataque protegido, investigación al Espía, restricción y expulsión del Bufón con continuación de partida.

Usar ejemplos controlados para el sorteo y registrar sus resultados; probar ausencia de ganador y final de partida. Estas son pruebas propuestas: no se han ejecutado porque aquí no existe todavía la implementación.

7. Instalación, ejecución y mantenimiento

La simulación requiere que primero se escriba el código Python y se definan su versión y dependencias. Una vez hecho, documentar los requisitos reales, instrucciones exactas de instalación y ejecución, estructura final y pruebas realizadas. Para la beta Android también faltan el repositorio de código, versión mínima de Android, forma de distribución, configuración de Firebase y procedimientos de compilación y despliegue. No se dispone de credenciales ni se deben incluir en el repositorio.

8. Pendientes para cerrar la versión definitiva

- Acordar las reglas abiertas y programar y probar la simulación.
- Obtener el código de la beta móvil para verificar arquitectura, versiones, instalación, seguridad y despliegue.
- Contrastar los diagramas marcados como borrador con la implementación.
- Agregar instrucciones reproducibles y resultados de pruebas reales cuando existan.